

CASE STUDY

NYC Airbnb Price Prediction

An XGBoost pricing model that doesn't just predict a number — it explains why, and earns production traffic gradually through a shadow → A/B → canary pipeline before it's trusted with real users.

- Model trained, benchmarked, and served behind a full serving stack — caching, batching, rollout, drift detection. The AWS infrastructure (EKS, RDS, ECR) was provisioned and run successfully via Terraform, then torn down afterward to avoid ongoing cloud cost; config is in place to redeploy with terraform apply.



01 Overview

NYC Airbnb Price Prediction takes a listing's structural attributes — bedrooms, borough, amenities, review scores — and returns a predicted nightly price with a per-feature SHAP explanation of what drove the number. The model is XGBoost, trained on 20,485 real

InsideAirbnb listings after removing luxury outliers and duplicate columns, reaching $R^2=0.824$ and a \$58 median error on a held-out test set. What makes this more than a notebook model is the serving shell around it: ONNX Runtime inference behind FastAPI, a Redis prediction cache, async batch scoring, PSI-based drift detection, and a shadow → A/B → canary rollout pipeline, so a retrained challenger has to prove itself on live traffic before it ever becomes the champion serving 100% of requests.

02 The Problem

Pricing an NYC listing by eye means comparing it to a handful of similar listings and guessing — missing structural signals that provably move price, like private-vs-shared bathroom status (the single strongest predictor in this dataset, gain=0.259) or distance from Midtown, and with no way to explain to a host why a suggested price is what it is. The harder problem sits one level up: once a pricing model is live, retraining it on fresh data and pushing the new version straight to every user is how a subtly-worse model quietly costs hosts money for weeks before anyone notices. This project treats both as first-class problems — SHAP explanations answer "why this price," and a shadow → A/B → canary rollout answers "is the new model actually better" before it touches full traffic.

03 Design Principles

- A prediction without an explanation isn't a pricing tool, it's a black box — every response can return per-feature SHAP attributions, not just a dollar figure.

- No model reaches 100% of traffic on faith — every promotion moves through shadow (silent comparison), A/B (live split), and canary (gradual rollout) before becoming champion.
- Target encodings are fit on training data only — letting neighbourhood_price_rank see test-set prices during fitting would leak the test distribution and inflate R^2 without teaching the model anything real.

04 Architecture Deep-Dive

How the 8 pipeline stages (see the project overview) group into four defensive layers.

01 Ingress & Serving

STAGES 1-2 · API GATEWAY, SERVING LAYER

Nginx handles SSL termination and per-IP rate limiting before anything touches application logic, same cheap-checks-first pattern as the other projects here. FastAPI runs behind Gunicorn with 2 workers, and — critically — the shadow/A-B/canary routing decision happens before inference, not after: the serving layer decides which model version handles a given request, then hands off to whichever champion or challenger that routing resolved to.

02 Features & Inference

STAGES 3-5 · FEATURE PIPELINE, MODEL INFERENCE, EXPLANATION

58 features feed the model — log-transforms, polynomial terms, and interaction features like accommodates $\times$ review_scores_rating, plus a neighbourhood_price_rank target encoding fit strictly on training data to avoid leakage. Inference runs through ONNX

Runtime rather than native XGBoost — 31x faster and it releases Python's GIL, so multiple Gunicorn workers actually run inference in parallel instead of queuing behind it. Every prediction can carry a SHAP TreeExplainer breakdown; nine non-user-controllable features (like `is_private_bath` and `minimum_nights_avg_ntm`) are deliberately excluded from the explanation UI since showing a user a lever they can't pull isn't useful.

03 Caching & Rollout

STAGES 6-7 · CACHING, ROLLOUT PIPELINE

About 40% of production requests are repeat queries, so an MD5-keyed Redis cache with a 5-minute TTL skips ONNX inference entirely on a hit. The same Redis instance backs an async batch queue via BRPOP, atomic enough that only one worker ever claims a given job. On the model side, a retrained challenger never jumps straight to serving everyone — it moves through shadow (runs silently alongside the champion, no user impact), then A/B (a real traffic split, compared on live metrics), then canary (5% → 25% → 50% → 100%, with rollback at any stage) before it's promoted to champion.

04 Drift & Observability

STAGE 8 · DRIFT & RETRAIN

A nightly GitHub Actions job checks Population Stability Index against the training baseline; PSI above 0.2 on any feature is treated as critical drift and kicks off an automatic retrain — download the latest snapshot, clean, engineer features, train, and gate the new model on beating the current champion's R^2 before it's even allowed to enter the rollout pipeline as a challenger. Three observability pipelines run in parallel from the same pod: structured

logs to Datadog, Prometheus metrics (latency histograms, cache hit rate, canary traffic split) to Grafana, and distributed traces to Tempo/Loki — with drift and alert events routed to Slack, critical severity going to #incidents and everything else to a default channel.

05 Production & Compliance

Deployment targets EKS on Kubernetes 1.35, rolled out via Helm with ArgoCD watching the chart for GitOps-driven syncs, and secrets pulled from AWS Secrets Manager through External Secrets Operator rather than static keys in the cluster. Every push to main runs the test suite, builds multi-arch Docker images, scans them with Trivy, pushes to ECR, and triggers the Helm upgrade. Pods run as non-root with read-only filesystems and dropped Linux capabilities; NetworkPolicy restricts egress to just Redis, DNS, and AWS. The AWS resources themselves — EKS, RDS, ECR — were provisioned with Terraform, run successfully, and later torn down specifically to stop paying for idle infrastructure between work sessions; every config needed to bring it back is already committed.

Deployment

EKS (Kubernetes 1.35) via Helm + ArgoCD GitOps — infra is currently torn down to avoid AWS credit burn; terraform apply in infra/aws/ redeploys it

Cache

Redis 7 with AOF persistence — prediction cache plus an async batch

Database

Postgres/SQLite for predictions, A/B results, and ground truth — S3 for model artifacts

Monitoring

structlog → Fluent Bit → Datadog, Prometheus + Grafana, Tempo + Loki

queue (BRPOP)

tracing, Slack drift/alert webhooks

Scaling

HPA 2–5 replicas on the API deployment

Availability

Shadow → A/B → canary rollout with health-gated rollback before any model reaches 100% of traffic

06 Results

Four models were benchmarked on the same held-out 4,097-listing test set: a dummy mean-predictor ($R^2=0.00$, the floor check), Ridge regression ($R^2=0.729$, chosen for interpretable coefficients despite weaker accuracy), Random Forest ($R^2=0.816$), and XGBoost ($R^2=0.824$, \$58 MAE, 23.6% MAPE) — the champion. XGBoost's training R^2 of 0.938 versus test R^2 of 0.824 looks like overfitting at a glance, but 5-fold cross-validation on the training data alone scored ≈ 0.81 — a 0.013 gap to the true unseen test score, which is the actual evidence the model generalizes rather than having memorized training leaves the way boosted trees structurally do. 131 automated tests cover the API contract, the A/B/canary state machine, and — specifically — that the feature list used at training time matches what the predictor expects at inference, a class of bug that fails silently otherwise.

- $R^2=0.824$ on a held-out test set of 4,097 listings
- \$58 median absolute error per prediction
- 58 engineered features across 20,485 core-market listings
- 131 automated tests in CI
- 205 commits building out the full MLOps lifecycle
- Shadow → A/B → canary rollout gates every model promotion

07 What's Next

The model's known ceiling without new data sources is roughly 82–85% R^2 — closing that gap needs signals this dataset doesn't have: historical booking/occupancy rate (worth an estimated +3–4% R^2), seasonal pricing, competitive density within 500m, and photo quality. It's also explicitly NYC-only (`neighbourhood_price_rank` and `dist_from_midtown` are meaningless elsewhere) and excludes the top 1% of listings by price, where predictions are unreliable without demand data. Infrastructure-wise, the EKS cluster is currently torn down between work sessions to avoid idle AWS cost — every Terraform and Helm config needed to bring it back is committed and ready.

08 Tech Stack

FastAPI

XGBoost

ONNX Runtime

Redis

Kubernetes

Terraform

MLflow

SHAP

Prometheus

Grafana

React